

Problème de couverture par cliques dans les graphes prismatiques

Piero Terrier¹, Cléophee Robin², and Thomas Bellitto³

¹ ENS Rennes

piero.terrier@ens-rennes.fr

² IRIF — Université Paris Cité

cleophee.robin@irif.fr

³ LIP6 — Sorbonne Université

thomas.bellitto@lip6.fr

1 Introduction

Présentation générale

Le problème central auquel nous nous intéressons dans ce rapport est celui de la *coloration des graphes*, qui consiste à attribuer une couleur à chaque sommet d'un graphe de façon que deux sommets adjacents reçoivent toujours des couleurs distinctes, tout en minimisant le nombre total de couleurs utilisées. Ce problème est fondamental en algorithmique et possède de nombreuses applications pratiques, notamment dans la planification, la gestion de ressources et les réseaux. Cependant, déterminer si un graphe peut être colorié avec k couleurs est un problème NP-complet pour tout $k \geq 3$ [5], ce qui signifie qu'il est peu probable d'obtenir un algorithme en temps polynomial pour le résoudre dans le cas général.

Pour mieux cerner les frontières entre les instances facilement résolubles et celles qui restent difficiles, il est naturel de restreindre l'étude du problème de coloration à certaines classes particulières de graphes. Parmi celles-ci, les *classes héréditaires*, définies comme des ensembles de graphes fermés par passage aux sous-graphes induits (sous-graphe obtenu par suppression de sommets), occupent une place centrale. En interdisant certaines structures comme sous-graphes induits, on obtient des familles partageant des propriétés communes qui peuvent parfois être exploitées pour concevoir des algorithmes polynomiaux.

Lorsque l'on interdit un seul sous-graphe induit, le problème de coloration est complètement caractérisé : il est polynomial si le graphe interdit est un sous-graphe induit de P_4 (un chemin de taille 4) ou de $P_3 + K_1$ (un chemin de taille 3 et un sommet isolé), et NP-complet dans les autres cas [6]. Cependant, dès que l'on interdit plusieurs sous-graphes simultanément, la question se complique considérablement et plusieurs cas demeurent encore ouverts.

Dans le cadre de ce stage, nous nous intéressons plus précisément aux graphes *antiprismatiques*, définis par l'exclusion des trois sous-graphes induits suivants : le graphe $K_{1,3}$ (un sommet relié à trois autres sommets), le graphe $K_2 + 2K_1$ (une arête et deux sommets isolés), et le graphe $4K_1$ (quatre sommets isolés).

La complexité du problème de coloration sur cette classe constitue l'un des trois derniers cas encore non résolus lorsque l'on interdit uniquement des graphes de taille quatre [8].

Pour aborder ce problème, il est utile d'observer que colorier un graphe G consiste à partitionner ses sommets en ensembles indépendants, c'est-à-dire en ensembles de sommets deux à deux non adjacents. On note $\overline{G}$ le graphe complémentaire de G , c'est à dire le graphe ayant les mêmes sommets que G et tel que deux sommets sont adjacents si et seulement si ils ne le sont pas dans G . Ces ensembles indépendants dans G deviennent des cliques dans $\overline{G}$, et une coloration de G se traduit ainsi en une couverture par cliques de $\overline{G}$. En d'autres termes, calculer le nombre chromatique de G revient à trouver une partition minimale des sommets de son complémentaire $\overline{G}$ en cliques.

Ce passage au complémentaire se justifie par la description plus simple des arêtes des graphes de la classe complémentaire des antiprismatiques, à savoir la classe des graphes *prismatiques*. Un graphe est dit prismatique si, pour tout triangle et tout sommet extérieur à ce triangle, ce sommet possède exactement un voisin dans le triangle. Cette contrainte locale forte confère à ces graphes une structure qui a permis à Chudnovsky et Seymour de les caractériser complètement (voir section 2). Nous nous appuyerons sur cette description détaillée pour tenter de concevoir un algorithme de couverture par cliques en temps polynomial, et ainsi résoudre indirectement le problème de coloration pour les graphes antiprismatiques.

Dans ce travail, nous nous intéressons plus particulièrement à certaines sous-classes des graphes prismatiques, et en particulier aux *core path of triangles graphs*. Nous donnons pour cette sous-classe un algorithme polynomial renvoyant une solution dont la taille diffère d'au plus 2 de celle d'une solution optimale.

Outline

Dans la prochaine section, nous présenterons la classification des graphes prismatiques, en mettant en évidence les cas traités et ceux qui restent ouverts. Dans la section 3, nous introduirons la notion de *path of triangles graphs*, avant de proposer pour les *core path of triangles graphs* un algorithme polynomial fournissant une solution approchée dont la taille diffère d'au plus deux de celle d'une solution optimale. Nous établirons ensuite sa correction. Puis, nous présenterons certains programmes ayant servis à tester et émettre des conjectures. Enfin, nous discuterons des perspectives offertes par ce travail et des directions possibles pour poursuivre l'étude de ces graphes et affiner les résultats obtenus.

2 État de l'art

Nous nous intéressons donc au problème de couverture par cliques dans les *graphes prismatiques*. Dans ces graphes, toute couverture par cliques se compose uniquement de triangles, d'arêtes et de sommets isolés, puisqu'ils ne contiennent pas de K_4 .

Un paramètre clé est $\Lambda(G)$, qui désigne la taille d'un *hitting set minimal* pour les triangles : un ensemble de sommets intersectant tous les triangles de G . Ce paramètre contrôle directement le nombre maximal de triangles disjoints, et permet ainsi de borner la complexité du problème. En particulier, lorsque $\Lambda(G)$ est borné par une constante, le problème de couverture par cliques peut être résolu en temps polynomial [7]. En effet, dans ce cas, on peut énumérer en temps polynomial toutes les façons de choisir un ensemble de triangles disjoints. Pour chacune de ces configurations, il suffit ensuite de déterminer un couplage maximum sur l'ensemble des sommets restants, opération qui s'effectue également en temps polynomial.

Les travaux de Chudnovsky et Seymour ([1] et [2]) fournissent une classification complète des graphes prismatiques, permettant ainsi d'identifier au fur et à mesure les sous-cas pour lesquels le problème se résout en temps polynomial, ainsi que ceux qui demeurent encore ouverts. Certaines définitions ne seront pas introduites car il s'agit juste ici de présenter une vue d'ensemble de la classification des prismatiques.

- Si le graphe est *non orientable*, soit il admet un hitting set de taille au plus 5, soit c'est un sous-graphe induit du graphe prismatique de Schläfli (et donc a au maximum 27 sommets) [7]. Ceci permet dans les deux cas de résoudre le problème en temps polynomial.
- Si le graphe est *orientable* et n'est pas 3-*colorable*, alors, d'après le lemme 11.2 de [1], soit il admet un hitting set de taille au plus 3, auquel cas on peut résoudre le problème en temps polynomial, soit il s'agit d'un *cycle of triangles graph*, dont la structure a été entièrement décrite par Chudnovsky et Seymour.
- Si le graphe est 3-*colorable*, alors soit il n'est pas premier et on peut le décomposer via une *worn chain decomposition*, soit il est premier. Dans ce dernier cas, le lemme 12.3 de [1] montre qu'il appartient alors à l'une des trois sous-classes :
 - $\mathcal{Q}_0$ (graphes sans triangles),
 - $\mathcal{Q}_1$ (le graphe $L(K_{3,3})$),
 - $\mathcal{Q}_2$ (les *path of triangles graphs*).

Là encore, $\mathcal{Q}_0$ et $\mathcal{Q}_1$ conduisent à des instances où le problème est résolu. Les *path of triangles graphs*, comme les *cycle of triangles graphs*, ont vu leur structure intégralement caractérisée par Chudnovsky et Seymour, mais le problème de couverture par cliques y reste ouvert. Leur structure étant quasiment identiques, l'étude de l'un apporte souvent également des résultats pour l'autre.

Cette situation est synthétisée dans le schéma ci-dessous issu de [7].

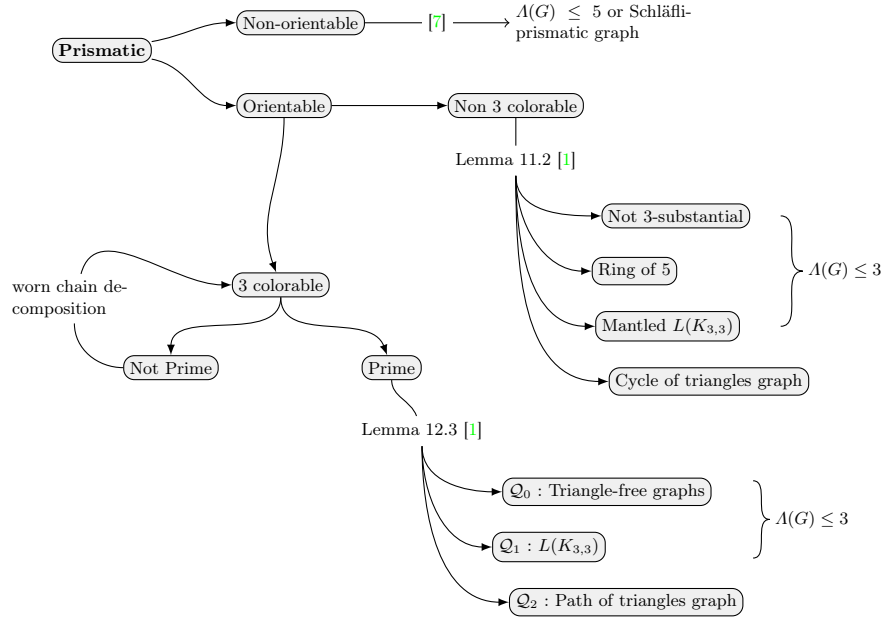


FIGURE 1 – Diagramme de la classification des graphes prismatiques

Ainsi, les seuls cas où le problème de couverture par cliques demeure non résolu concernent précisément les *path of triangles graphs*, les *cycle of triangles graphs*, ainsi que les graphes nécessitant une *worn chain decomposition*, ouvrant ainsi la voie à une étude plus approfondie de ces différentes configurations.

3 Contenu

3.1 Définitions et notations

Nous présentons ici la définition des *paths of triangles graphs* telle qu'énoncée par Chudnovsky et Seymour [1].

On dit qu'un graphe G est *recouvert par des triangles* si chaque sommet de G appartient à un triangle. De plus, G est *triangle-connexe* si l'on ne peut pas partitionner $V(G)$ en deux sous-ensembles A et B , chacun contenant au moins un triangle, de telle sorte que chaque triangle de G soit entièrement inclus dans A ou dans B .

Definition 1. *Un graphe G est un Q_i s'il peut être construit selon la procédure suivante, en quatre étapes :*

Étape 1 : Construction d'une chaîne linéaire

— Soit S un chemin avec comme sommets $s_1, \dots, s_n$ dans l'ordre.



FIGURE 2 – chemin

— Soit $W \subseteq V(S)$ un stable, tel que $s_1, s_n \notin W$.

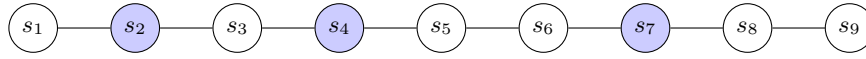


FIGURE 3 – Les sommets en bleu forment le stable $W = \{s_2, s_4, s_7\}$

— Pour chaque $s_i \in W$, on duplique s_i autant de fois que souhaité : on ajoute un ensemble de nouveaux sommets, chacun ayant les mêmes voisins que s_i . On note $\hat{X}_{2i}$ l'ensemble composé de s_i et de ses copies. Pour chaque $s_i \notin W$, on pose $\hat{X}_{2i} = \{s_i\}$.

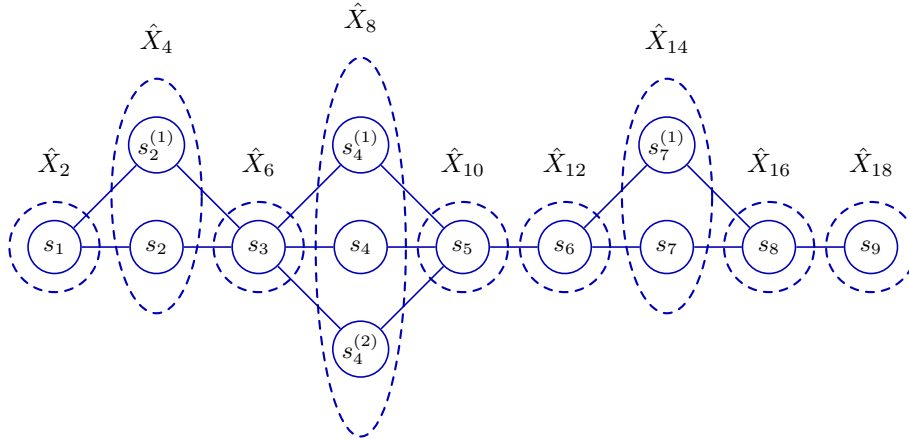


FIGURE 4 – duplication des sommets du stable

— Pour chaque arête uv de ce graphe, on ajoute un sommet w adjacent uniquement à u et v . L'ensemble des sommets ainsi ajoutés entre $\hat{X}_{2i}$ et $\hat{X}_{2i+2}$ (pour $1 \leq i < n$) est noté M_{2i+1} . On pose $M_1 = M_{2n+1} = \emptyset$.

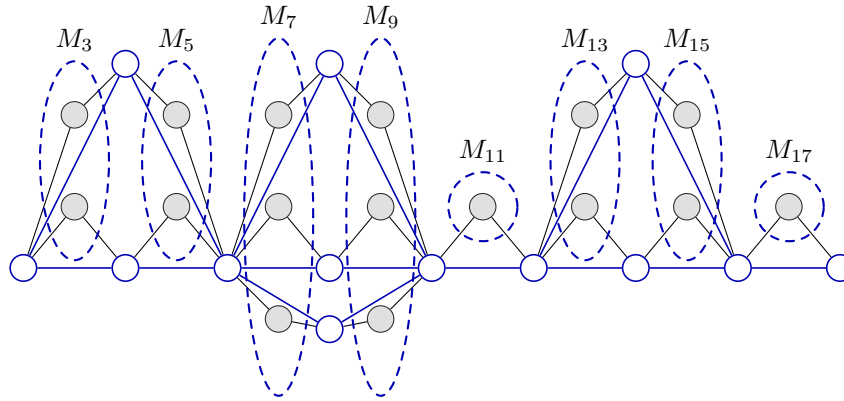


FIGURE 5 – ajout des sommets dans M_i

- Pour chaque $s_i \notin W$, on ajoute autant de paires de sommets adjacents $\{x, y\}$ que souhaité, chacun étant adjacent uniquement à s_i et à l'autre. On note R_{2i-1} l'ensemble des nouveaux voisins de s_i du côté gauche (x) et L_{2i+1} ceux du côté droit (y). On pose $R_{2i-1} = L_{2i+1} = \emptyset$. On impose aussi $R_1 = L_{2n+1} = \emptyset$.

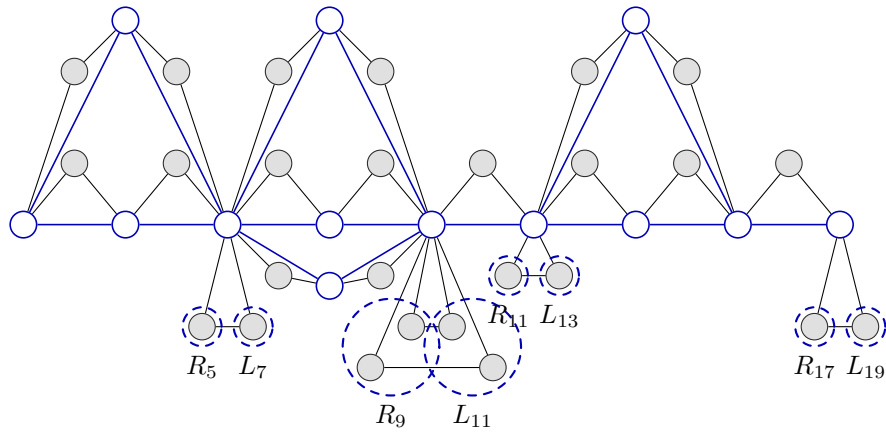


FIGURE 6 – ajout des sommets dans R_i et L_i

On appelle le graphe obtenu une chaîne linéaire.

Étape 2 : Ajout de sommets dans les boîtes paires

- Pour chaque $1 \leq i \leq n$, on définit un ensemble $X_{2i} \supseteq \hat{X}_{2i}$ de sommets, où les sommets de $X_{2i} \setminus \hat{X}_{2i}$ sont nouveaux. Les ensembles $X_2, X_4, \dots, X_{2n}$ sont disjoints deux à deux.
- Chaque sommet $w \in X_{2i} \setminus \hat{X}_{2i}$ est adjacent uniquement à des sommets dans $R_{2i-1} \cup L_{2i+1}$, et pour chaque arête entre R_{2i-1} et L_{2i+1} , w est adjacent à exactement une de ses extrémités.

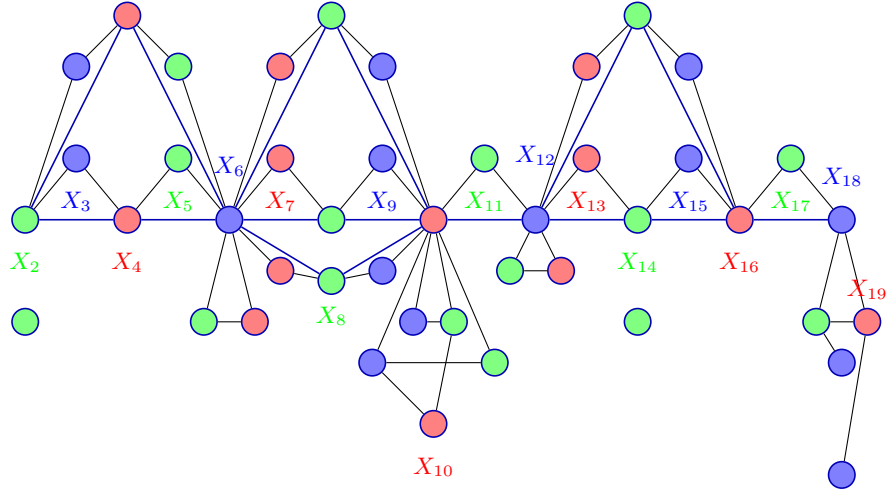


FIGURE 7 – ajout des sommets dans $X_{2i} \setminus \hat{X}_{2i}$

Étape 3 : Ajout d'arêtes longues

- Pour toute paire de sommets $u \in X_i$ et $v \in X_j$ avec $1 \leq i < j \leq 2n+1$ et $j \geq i+2$, on ajoute une arête uv si l'une des conditions suivantes est satisfaite :
 - $j \geq i+3$ et $j-i \equiv 2 \pmod{3}$;
 - $j = i+2$ et i est pair ;
 - $j = i+2$, i est impair, et soit $u \notin R_i$, soit $v \notin L_{i+2}$, et u, v n'ont pas de voisin commun dans $\hat{X}_{i+1}$.

Remarque : les nouvelles arêtes ne forment pas de nouveaux triangles. Le graphe devenant trop dense à partir de cette étape, on ne le représentera plus.

Étape 4 : Suppression d'arêtes entre boîtes paires

- Pour tous i, j pairs avec $2 \leq i < j \leq 2n$, on peut supprimer arbitrairement les arêtes entre $X_i \setminus \hat{X}_i$ et $X_j \setminus \hat{X}_j$.

Le graphe final obtenu est appelé un . Les triangles dans ce graphe sont exactement ceux présents à la fin de l'étape 1.

De manière équivalente, on dit que G est un path of triangles graph s'il existe un entier $n \geq 1$ et des sous-ensembles indépendants deux à deux disjoints

$$X_1, X_2, \dots, X_{2n+1}$$

dont la réunion est $V(G)$, satisfaisant les conditions suivantes :

(P1) Pour tout $1 \leq i \leq n$, il existe un sous-ensemble non vide $\hat{X}_{2i} \subseteq X_{2i}$ tel que $|\hat{X}_2| = |\hat{X}_{2n}| = 1$, et pour tout $0 < i < n$, au moins un des ensembles $\hat{X}_{2i}$ ou $\hat{X}_{2i+2}$ est de cardinal 1.

(P2) Pour tout $1 \leq i < j \leq 2n+1$:

1. Si $j - i \equiv 2 \pmod{3}$ et qu'il existe $u \in X_i$ et $v \in X_j$ non adjacents, alors soit i et j sont impairs et $j = i+2$, soit i et j sont pairs et $u \notin \hat{X}_i$ et $v \notin \hat{X}_j$.
2. Si $j - i \not\equiv 2 \pmod{3}$, alors soit $j = i+1$, soit X_i est anti-complet à X_j .

(P3) Pour tout $1 \leq i \leq n+1$, l'ensemble X_{2i-1} est la réunion de trois ensembles deux à deux disjoints L_{2i-1} , M_{2i-1} et R_{2i-1} , avec

$$L_1 = M_1 = M_{2n+1} = R_{2n+1} = \emptyset.$$

(P4) Si $R_1 = \emptyset$, alors $n \geq 2$ et $|\hat{X}_4| > 1$. De même, si $L_{2n+1} = \emptyset$, alors $n \geq 2$ et $|\hat{X}_{2n-2}| > 1$.

(P5) Pour tout $1 \leq i \leq n$:

$$X_{2i} \text{ est anti-complet à } L_{2i-1} \cup R_{2i+1},$$

et

$$X_{2i} \setminus \hat{X}_{2i} \text{ est anti-complet à } M_{2i-1} \cup M_{2i+1}.$$

De plus, chaque sommet de $X_{2i} \setminus \hat{X}_{2i}$ est adjacent à exactement une extrémité de chaque arête entre R_{2i-1} et L_{2i+1} .

(P6) Pour tout $1 \leq i \leq n$, si $|\hat{X}_{2i}| = 1$, alors :

1. R_{2i-1} et L_{2i+1} sont couplés, et chaque arête entre $M_{2i-1} \cup R_{2i-1}$ et $L_{2i+1} \cup M_{2i+1}$ est entre R_{2i-1} et L_{2i+1} .
2. Le sommet de $\hat{X}_{2i}$ est complet à $R_{2i-1} \cup M_{2i-1} \cup L_{2i+1} \cup M_{2i+1}$.
3. L_{2i-1} est complet à X_{2i+1} et X_{2i-1} est complet à R_{2i+1} .
4. Si $i > 1$, alors M_{2i-1} et $\hat{X}_{2i-2}$ sont couplés, et si $i < n$, alors M_{2i+1} et $\hat{X}_{2i+2}$ sont couplés.

(P7) Pour tout $1 < i < n$, si $|\hat{X}_{2i}| > 1$, alors :

1. $R_{2i-1} = L_{2i+1} = \emptyset$;
2. si $u \in X_{2i-1}$ et $v \in X_{2i+1}$, alors u et v ne sont pas adjacents si et seulement s'ils ont le même voisin dans $\hat{X}_{2i}$.

On remarque qu'un sommet de G n'appartient à aucun triangle de G si et seulement s'il appartient à l'un des ensembles $X_{2i} \setminus \hat{X}_{2i}$. Donc, si pour tout i on a $\hat{X}_{2i} = X_{2i}$, G est recouvert par des triangles, et on dit alors que G est un core path of triangles graph.

Chemins de couleurs On introduit une classe de graphe que l'on appellera *chemins de couleurs*, qui contiendra en particulier certains sous-graphes induits des *path of triangles graphs*, et qui nous sera utile afin de prouver la correction de notre algorithme.

Soit un entier $n \geq 1$ et une fonction de coloration :

$$c : \llbracket 1, n \rrbracket \rightarrow \llbracket 0, 2 \rrbracket.$$

On définit le graphe $G_c = (V, E)$ de la manière suivante :

- L'ensemble des sommets est $V = \llbracket 1, n \rrbracket$;
- L'ensemble des arêtes est

$$E = \{ \{i, j\} \mid i < j, (c(i), c(j)) \in \{(0, 1), (1, 2), (2, 0)\} \}.$$

Notation. On introduit les sous-ensembles de sommets :

$$\begin{aligned} V_0 &= \{i \in V \mid c(i) = 0\}, \\ V_1 &= \{i \in V \mid c(i) = 1\}, \\ V_2 &= \{i \in V \mid c(i) = 2\}. \end{aligned}$$

Remarque. On a la partition :

$$V = V_0 \sqcup V_1 \sqcup V_2.$$

Les ensembles V_0 , V_1 et V_2 sont des stables, et le graphe G_c est sans triangles.

Definition 2. On appelle chemins de couleurs l'ensemble de graphes $\mathcal{G}$ constitué de tous les graphes obtenus à partir des graphes G_c en supprimant arbitrairement des arêtes de la forme $\{i, i+1\}$. Autrement dit :

$$\begin{aligned} \mathcal{G} = \{ (V(G_c), E(G_c) \setminus R) \mid & c : \llbracket 1, n \rrbracket \rightarrow \llbracket 0, 2 \rrbracket, \\ & A \subset \llbracket 1, n-1 \rrbracket, \\ & R = \{ \{i, i+1\} \mid i \in A \} \}. \end{aligned}$$

3.2 Algorithme d'approximation

Algorithme 1 : Approximation de couverture par cliques minimale sur un *core path of triangles graph*

Entrée : Un *core* G et sa décomposition $(X_1, X_2, \dots, X_{2n+1})$
Sortie : Une couverture par cliques dont la taille diffère d'au plus 2 d'une couverture par cliques minimale

```

1 pour  $i$  allant de 1 à  $n$  faire
2   Sélectionner un sommet  $x_i \in X_{2i}$ ;
   //  $(x_1, x_2, \dots, x_n)$  est un chemin
3  $T \leftarrow \emptyset$ ;
   // ensemble des triangles choisis
4 pour  $i$  allant de 1 à  $n$  faire
5   si  $x_i$  n'appartient à aucun triangle de  $T$  alors
6     si il existe  $y, z$  hors du chemin tels que  $\{x_i, y, z\}$  forme un
       triangle alors
7        $T \leftarrow T \cup \{\{x_i, y, z\}\}$ ;
8     sinon
9       // alors nécessairement  $i \neq n$  et il existe  $y$  tel que
        $\{x_i, x_{i+1}, y\}$  forme un triangle
        $T \leftarrow T \cup \{\{x_i, x_{i+1}, y\}\}$ ;
10  $G' \leftarrow G$  privé des sommets des triangles de  $T$ ;
11  $M \leftarrow \text{CouplageMaximum}(G')$ ;
12  $S \leftarrow$  sommets non appariés par  $M$  dans  $G'$ ;
Renvoyer :
13 couverture constituée des triangles  $T$ , des arêtes  $M$  et des sommets
    isolés de  $S$ ;
```

3.3 Correction de l'algorithme

Nombre maximum de triangles disjoints

Proposition 1 À la fin de l'algorithme, T est un ensemble de triangles disjoints de taille maximale.

Voir la [démonstration](#) en annexe.

Couplage Pour établir que la couverture renvoyée par l'algorithme est *suffisamment proche* de la solution optimale, il suffit d'exhiber un couplage dans G' laissant *peu de sommets isolés*.

En effet, la taille de la couverture est donnée par :

$$|V(G)| - 2|T| - |M|$$

et minimiser cette quantité revient à maximiser $2|T| + |M|$. Comme nous avons déjà choisi un nombre maximum de triangles disjoints, obtenir une meilleure solution revient à pouvoir augmenter le nombre d'arêtes du couplage.

Soient :

- k le nombre de triangles en moins d'une autre solution,
- c la diminution du nombre total de cliques,
- s le nombre de sommets non appariés par M .

Alors, retirer k triangles libère $3k$ sommets, et l'on peut former au plus :

$$\left\lfloor \frac{3k + s}{2} \right\rfloor$$

nouvelles arêtes.

Pour obtenir une solution avec c cliques de moins, il faut :

- compenser la perte des k triangles en ajoutant $2k$ arêtes (car chaque triangle compte pour deux arêtes dans $2|T| + |M|$),
- puis ajouter encore c arêtes supplémentaires pour réduire la couverture de c cliques.

Ainsi, il faut pouvoir former au total $2k + c$ nouvelles arêtes grâce aux sommets libérés.

On en déduit la condition :

$$\left\lfloor \frac{3k + s}{2} \right\rfloor \geq 2k + c.$$

Comme $2k + c$ est un entier, on a l'équivalence :

$$\left\lfloor \frac{3k + s}{2} \right\rfloor \geq 2k + c \iff \frac{3k + s}{2} \geq 2k + c.$$

D'où finalement :

$$3k + s \geq 4k + 2c \iff s \geq k + 2c.$$

Cette inégalité montre explicitement comment le nombre de sommets non appariés s contrôle la possibilité d'améliorer la couverture en réduisant le nombre de cliques de c .

Ainsi :

- si $s = 0$ ou 1 , la solution est *optimale*,
- si $s = 2$, on peut parfois l'améliorer de 1 sans réduire le nombre de triangles mais en les prenant d'une manière différente,
- si $s = 3$, on peut améliorer la solution de 1 en prenant au plus un triangle de moins que le nombre maximum,

- si $s = 4$, on peut l'améliorer de 1 avec au plus deux triangles de moins que le nombre maximum, et même de 2 en prenant le nombre maximum de triangles disjoints d'une manière différente.

Nous exhiberons dans la suite un couplage dans G' garantissant au maximum 4 sommets isolés. Comme notre algorithme choisit un couplage maximum, celui-ci est au moins aussi grand que celui que nous exhiberons. Ainsi, la taille de notre couverture différera toujours de moins de 2 de celle d'une couverture minimale.

Proposition 2 *Il existe un couplage M_0 dans G' tel que le sous-graphe induit de G' obtenu en enlevant les sommets de M_0 soit un chemin de couleurs tel que deux sommets consécutifs ne soient jamais de la même couleur.*

Voir la [démonstration](#) en annexe.

Définition 1 *On note $\mathcal{H}$ l'ensemble des graphes G tels que, pour tout sous-graphe induit G' de G , on ait*

$$|M^*| = \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right),$$

où :

- M^* est un couplage maximum de G' ;
- n le nombre de sommets de G' ;
- V_c^* une couverture par sommets minimale de G' .

Définition 2 *On définit*

$$\mathcal{F} = \text{Free}(\{C_{2k+1} + K_1 : k \in \mathbb{N}^*\}),$$

c'est-à-dire la classe des graphes ne contenant aucun sous-graphe induit qui soit l'union disjointe d'un cycle impair et d'un sommet isolé.

Théorème 1 *On a l'égalité :*

$$\mathcal{H} = \mathcal{F}.$$

Voir la [démonstration](#) en annexe.

Proposition 3 *On a l'inclusion :*

$$\mathcal{G} \subset \mathcal{F}.$$

Voir la [démonstration](#) en annexe.

Proposition 4 *Dans un chemin de couleurs G avec n sommets où deux sommets consécutifs ne sont jamais de la même couleur :*

$$|S^*| \leq \left\lfloor \frac{n}{2} \right\rfloor + 2$$

où S^* est un stable maximum de G .

Voir la [démonstration](#) en annexe.

Théorème 2 *Il existe un couplage de G' laissant au plus 4 sommets isolés.*

Voir la [démonstration](#) en annexe.

Conclusion L'analyse précédente nous permet de conclure que la couverture renvoyée par notre algorithme diffère d'au plus deux en taille par rapport à une couverture minimale, ce qui prouve la correction de notre algorithme. De plus, la construction de T se fait en temps polynomial, et un couplage maximum dans un graphe quelconque se calculant également en temps polynomial [3], on en conclut que notre algorithme est bien polynomial.

3.4 Programmes C++

Afin de tester certaines conjectures et de pouvoir en émettre d'autres, 4 programmes ont été réalisés en C++.

Programme 1 Voir le [code](#) en annexe.

Ce programme calcule de manière exacte la couverture par cliques minimale pour certains *core path of triangles graphs*. Il procède en testant toutes les façons possibles de sélectionner des triangles disjoints, en commençant par celles de cardinalité maximale. Pour chaque configuration de triangles disjoints, il calcule un couplage maximum sur les sommets restants. L'algorithme s'arrête dès qu'un certificat d'optimalité est obtenu. Par exemple, si une solution utilise le nombre maximal de triangles disjoints et qu'un couplage maximum laisse au plus un sommet isolé. Ou encore si, après avoir exploré toutes les configurations avec le nombre maximal de triangles, l'une d'elles a un couplage laissant exactement deux sommets isolés.

Dans la majorité des cas, ces heuristiques permettent de trouver rapidement une solution optimale, car la plupart des graphes que nous testons admettent une solution utilisant le nombre maximal de triangles disjoints.

Le programme prend en entrée un entier n , représentant la taille du chemin initial, suivi de n valeurs binaires (0 ou 1), où chaque 1 indique la présence d'un *triangle pendant* attaché au sommet correspondant dans le chemin. Une variable *choix*, fixée à 0, 1 ou 2, permet de contrôler la sortie du programme :

- 0 : affiche la solution complète (taille, triangles et arêtes sélectionnés) ;
- 1 : affiche le graphe lorsque la solution n'utilise pas le nombre maximal de triangles disjoints, ainsi que le nombre de triangles et de sommets isolés ;
- 2 : n'affiche que la taille de la couverture minimale.

Ce programme peut être utilisé à la main pour des graphes spécifiques ou intégré dans une procédure de test automatique.

Programme 2 Voir le [code](#) en annexe.

Ce programme est similaire au précédent, à la différence qu'il ne renvoie pas la solution optimale. Il implémente l'algorithme d'approximation présenté en pseudo-code (Algorithme 1), mais restreint à une sous-classe spécifique des *core path of triangles graphs*, à savoir les chemins de triangles avec toutes les boîtes X_{2i} de cardinal 1, et avec au plus un *triangle pendant* par sommet du chemin. Il ne propose pas le choix 1 du programme précédent ; ainsi, le choix 1 correspond ici à l'ancien choix 2.

Ce programme peut être utilisé à la main pour tester l'algorithme d'approximation sur une instance particulière du problème, ou intégré dans une procédure de test automatique.

Programme 3 Voir le [code](#) en annexe.

Ce programme sert de générateur de tests. En fonction de la valeur de la variable *choix*, il permet de réaliser soit des tests aléatoires, soit des tests exhaustifs. Il est conçu pour être utilisé en combinaison, via un pipe, avec les Programmes 1 et 2. Une option permet également de comparer directement les résultats des deux programmes et d'identifier les graphes pour lesquels leur réponse diffèrent.

Combiné au Programme 1 avec le choix 1, ce générateur de tests a permis de mettre en évidence l'existence de nombreux graphes pour lesquels prendre le nombre maximal de triangles disjoints n'est pas optimal, quelle que soit la manière dont on les sélectionne. Il a également révélé l'existence de graphes dont la solution optimale utilise deux triangles de moins que le maximum possible, mais uniquement pour les chemins de taille congrue à 1 modulo 3 (13, 16, 19, 22).

Enfin, ce générateur de tests n'a identifié aucun cas où l'algorithme d'approximation renvoyait une solution de taille exactement deux supérieure à celle d'une couverture par cliques minimale. Cela permet d'espérer que notre algorithme puisse en réalité toujours renvoyer une solution de taille au plus 1 plus grande qu'une solution optimale, bien que nous n'ayons démontré qu'une approximation de 2.

Programme 4 Voir le [code](#) en annexe.

Ce programme vérifie l'égalité entre la taille d'un couplage maximum et celle d'une couverture par sommets minimale dans les *chemins de couleurs*. Il effectue une génération exhaustive de tous les *chemins de couleurs* jusqu'à une taille donnée, en vérifiant également si le graphe sous-jacent est un chemin.

Comme les *chemins de couleurs* forment une classe héréditaire de graphes, le programme a permis d'établir qu'aucun *chemin de couleurs* n'a de chemin induit de taille 7. De plus, l'égalité ayant été vérifiée pour tous les *chemins de couleurs* de taille inférieure à 8, cela fournit une preuve numérique de la Proposition 3.

4 Conclusions et perspectives

En résumé, nous avons proposé un algorithme fournissant une approximation à deux unités près de la solution optimale pour les *core path of triangles graphs*. Les résultats expérimentaux suggèrent toutefois que cet algorithme pourrait en réalité garantir une approximation à un près.

Ce résultat pourrait vraisemblablement être généralisé, avec peu d'adaptations, à la classe des *core cycle of triangles graphs*, dont la structure est quasiment identique.

Pour aller au-delà de l'approximation et obtenir un algorithme exact, il serait intéressant d'étudier la *cliquewidth* des *core path of triangles graphs*. Si cette largeur de clique est bornée, alors le problème de couverture par cliques peut être résolu en temps polynomial [4]. Si ce n'est pas le cas, on devrait pouvoir espérer que ça le devienne en bornant la taille des ensembles X_{2i} . Cela permettrait de supposer que l'on dispose d'un bloc X_{2i} de cardinal arbitrairement grand, offrant ainsi un grand nombre de sommets des trois couleurs des boîtes X_{2i-1} , X_{2i} et X_{2i+1} . Cela pourrait alors garantir l'existence d'un couplage presque parfait, fournissant ainsi un certificat d'optimalité.

Les programmes peuvent être généralisés pour pouvoir tester toute la classe des *core path of triangles graphs*, et ainsi permettre d'établir de nouvelles conjectures.

En revanche, si l'on s'intéresse aux s dans leur entiereté, le problème semble devenir nettement plus complexe. Une tentative de preuve de NP-complétude pourrait alors être envisagée.

Références

1. Chudnovsky, M., Seymour, P. : Claw-free graphs. I. Orientable prismatic graphs. *Journal of Combinatorial Theory, Series B* **97**(6), 867–903 (2007). <https://doi.org/10.1016/j.jctb.2007.02.002>, <http://linkinghub.elsevier.com/retrieve/pii/S0095895607000172>
2. Chudnovsky, M., Seymour, P. : Claw-free graphs. II. Non-orientable prismatic graphs. *Journal of Combinatorial Theory, Series B* **98**(2), 249–290 (2008). <https://doi.org/10.1016/j.jctb.2007.06.006>, <http://linkinghub.elsevier.com/retrieve/pii/S0095895607000780>
3. Edmonds, J. : Paths, trees, and flowers. *Canadian Journal of Mathematics* **17**(3), 449–467 (1965). <https://doi.org/10.4153/CJM-1965-045-4>
4. Espelage, W., Gurski, F., Wanke, E. : How to solve np-hard graph problems on clique-width bounded graphs in polynomial time. In : Brandstädt, A., Le, V.B. (eds.) *Graph-Theoretic Concepts in Computer Science (WG 2001)*, *Lecture Notes in Computer Science*, vol. 2204, pp. 117–128. Springer, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-45477-2_12
5. Karp, R.M. : Reducibility among combinatorial problems. In : Miller, R.E., Thatcher, J.W. (eds.) *Complexity of Computer Computations*, pp. 85–103. Springer (1972). https://doi.org/10.1007/978-1-4684-2001-2_9
6. Král', D., Kratochvíl, J., Tuza, Z., Woeginger, G. : Complexity of coloring graphs without forbidden induced subgraphs. In : Brandstädt, A., Le, V. (eds.) *Graph-Theoretic Concepts in Computer Science*. pp. 254–262. Springer Berlin Heidelberg, Berlin, Heidelberg (2001)
7. Preissmann, M., Robin, C., Trotignon, N. : On the complexity of colouring anti-prismatic graphs. *Algorithmica* **83**(2), 589–612 (2021). <https://doi.org/10.1007/s00453-020-00767-7>, <https://doi.org/10.1007/s00453-020-00767-7>
8. V. Lozin, D. Malyshev : Vertex coloring of graphs with few obstructions. *Discrete Applied Mathematics* pp. 273–280 (2015)

A Annexes techniques

A.1 Démonstrations

Démonstration 1 (Preuve de la Proposition 1)

Comme G est un core, pour tout $i \in \llbracket 1, n \rrbracket$, on a $X_{2i} = \hat{X}_{2i}$.
 Par (P1), les boîtes X_{2i} sont non vides, donc les sommets x_i sont bien définis.
 Par (P2)(1), X_{2i} et X_{2i+2} sont complets entre eux, donc $\{x_i, x_{i+1}\} \in E(G)$.
 Par (P1) et (P6)(2,4), pour tout $i \in \llbracket 1, n-1 \rrbracket$, l'une des deux boîtes X_{2i} ou X_{2i+2} est de cardinal 1 et est complète à M_{2i+1} , laquelle est couplée à l'autre boîte. En particulier, M_{2i+1} est non vide. On en déduit qu'il existe $y \in M_{2i+1}$ tel que $\{x_i, x_{i+1}, y\}$ forme un triangle.

D'après le lemme 3.1 de [7], si A est un triangle de G , alors il existe $i \in \llbracket 1, n \rrbracket$ tel que :

$$A \subset R_{2i-1} \cup X_{2i} \cup L_{2i+1} \quad \text{ou} \quad A \subset X_{2i-2} \cup M_{2i-1} \cup X_{2i},$$

avec un sommet dans chacune des trois boîtes, puisque celles-ci sont des stables.

Claim 1 Supposons qu'il existe X_{2i} tel que $|X_{2i}| > 1$. Alors à l'itération i , un triangle est ajouté et est de la forme $\{x_i, x_{i+1}, y\}$ avec $y \in M_{2i+1}$.

Preuve du claim. Soit $z \in X_{2i}$ tel que $z \neq x_i$. Comme x_i est un élément quelconque de X_{2i} , z vérifie les mêmes propriétés, donc il existe $y \in M_{2i-1}$ tel que $\{x_{i-1}, z, y\}$ forme un triangle.

On en déduit que x_i n'a pas pu être ajouté lors de l'itération $i-1$, car z et y vérifient la condition du si, ni avant, par la caractérisation des triangles. Un nouveau triangle doit donc être ajouté à l'itération i .

Or, par (P7)(1), on a $R_{2i-1} = L_{2i+1} = \emptyset$. Et par (P1), $1 < i < n$, $X_{2i-2} = \{x_{i-1}\}$ et $X_{2i+2} = \{x_{i+1}\}$. Il n'existe donc pas deux sommets en dehors du chemin pouvant former un triangle avec x_i .

Par conséquent, à l'itération i , le triangle ajouté est nécessairement de la forme $\{x_i, x_{i+1}, y\}$ avec $y \in M_{2i+1}$. $\square$

Montrons que l'assertion mentionnée dans le commentaire du sinon est bien vérifiée.

Par (P1), on a $|X_{2n}| = 1$.

Par (P4) :

- Soit $L_{2n+1} \neq \emptyset$, auquel cas, par (P6)(1,2), il existe $y \in R_{2n-1}$ et $z \in L_{2n+1}$ tels que $\{x_n, y, z\}$ forme un triangle.
- Soit $n \geq 2$ et $|X_{2n-2}| > 1$, auquel cas un triangle contenant x_{n-1} et x_n a été ajouté à l'itération $n-1$ par le claim 1.

On en déduit que l'assertion mentionnée dans le commentaire du `sinon` est bien vérifiée.

Montrons que T est un ensemble de triangles disjoints.

Soient A_1 et A_2 deux triangles dans T , où A_1 est ajouté à l'itération i de la boucle, et A_2 à l'itération j , avec $i < j$. On remarque que $x_i \in A_1$ et $x_j \in A_2$. De plus, par la première condition de la boucle, si $x_{i+1} \in A_1$, alors $j > i + 1$, ce qui implique $x_j \notin A_1$. De même, comme A_2 ne peut contenir de sommet du chemin autre que x_j ou x_{j+1} , on a aussi $x_i \notin A_2$.

Cas 1. Supposons $A_1 \subset R_{2i-1} \cup X_{2i} \cup L_{2i+1}$. Alors A_2 ne peut contenir :

- ni de sommet dans R_{2i-1} ou L_{2i+1} ,
- ni x_i .

Ainsi, A_1 et A_2 sont disjoints.

De même si A_2 a un seul sommet dans une boîte paire.

On suppose donc désormais que A_1 et A_2 ont tous deux deux sommets dans une boîte paire.

Cas 2. Supposons que A_1 a un sommet dans X_{2j} . Alors $j = i + 1$ et ce sommet est distinct de x_j , donc $|X_{2j}| > 1$. Donc par le [claim 1](#) :

$$A_2 \subset X_{2j} \cup M_{2j+1} \cup X_{2j+2}.$$

Et donc $A_1 \cap A_2 = \emptyset$.

De même si A_2 contient un sommet dans X_{2i} , A_1 et A_2 sont également disjoints.

Cas 3. Supposons $i = j + 2$ et que A_1 et A_2 aient un sommet dans X_{2i+2} . Le sommet de A_2 dans X_{2i+2} n'étant pas x_{i+1} , on a $|X_{2i+2}| > 1$. Donc d'après le [claim 1](#), x_{j-1} et x_j sont dans le triangle ajouté à l'itération $j - 1$, ce qui est impossible, car dans ce cas aucun triangle n'aurait dû être ajouté à l'itération j .

Cas 4. Les derniers cas restants sont :

- $i = j + 1$, $A_1 \subset X_{2i-2} \cup M_{2i-1} \cup X_{2i}$ et $A_2 \subset X_{2j} \cup M_{2j+1} \cup X_{2j+2}$;
- $i = j + 2$ et ($A_1 \subset X_{2i-2} \cup M_{2i-1} \cup X_{2i}$ ou $A_2 \subset X_{2j} \cup M_{2j+1} \cup X_{2j+2}$) ;
- $j > i + 2$.

Dans chacun de ces cas, les ensembles contenant les sommets de A_1 et A_2 sont disjoints, donc $A_1 \cap A_2 = \emptyset$.

On conclut donc que tous les triangles de T sont bien disjoints.

Montrons que le nombre de triangles dans T est maximal.

Comme tous les triangles ont au moins un sommet dans un hitting set des triangles, la taille d'un ensemble de triangles disjoints est bornée par la taille d'un hitting set. Exhiber un hitting set de la même taille que T nous permettra alors de conclure que T est un ensemble maximum de triangles disjoints.

On remarque que tous les sommets x_i appartiennent à un triangle de T . En effet, à l'itération i , soit x_i appartient déjà à un triangle de T , soit on ajoute dans T un triangle le contenant à cette étape. De plus tous les triangles ont soit un sommet du chemin x_i si ils ont été ajoutés dans le **si**, soit deux sommets du chemin x_i et x_{i+1} si ils ont été ajoutés dans le **sinon**.

Pour chaque triangle $A \in T$, on choisit le sommet $x_i \in A$ du chemin avec i le plus grand possible. Montrons que cet ensemble H est un hitting set des triangles, ce qui nous permettra de conclure, comme ce dernier a la même taille que T .

Soit A un triangle de G .

Cas 1 : A a un seul sommet dans une boîte paire X_{2i} .

Alors, par la caractérisation des triangles et la propriété (P7)(1), on a nécessairement $|X_{2i}| = 1$, donc $x_i \in A$.

Comme A est un triangle qui vérifie la condition du **si**, cela signifie que x_i a été ajouté dans un triangle dans le **si**, donc $x_i \in H$.

Cas 2 : A a deux sommets dans des boîtes paires X_{2i} et X_{2i+2} .

Trois sous-cas sont alors possibles :

- **Sous-cas 1 :** $|X_{2i}| = |X_{2i+2}| = 1$.
Alors $x_i \in A$ et $x_{i+1} \in A$, donc l'un des deux au moins appartient à H .
- **Sous-cas 2 :** $|X_{2i}| > 1$.
Par (P1), on a $|X_{2i+2}| = 1$, donc $x_{i+1} \in A$.
Or, d'après le claim 1, il existe un triangle dans T contenant x_i et x_{i+1} , donc $x_{i+1} \in H$.
- **Sous-cas 3 :** $|X_{2i+2}| > 1$.
Par (P1), on a $|X_{2i}| = 1$, donc $x_i \in A$.
Par le claim 1, on déduit que $x_{i+1} \notin A$, donc A vérifie la condition du **si**.
Par conséquent, x_i a été ajouté dans un triangle dans le **si**, et appartient donc à H .

On en conclut que T est bien un ensemble maximum de triangles disjoints.

[Retour à l'énoncé de la proposition.](#)

Démonstration 2 (Preuve de la Proposition 2)

On va construire un couplage M_0 de sorte de ne laisser des sommets que dans les ensembles M_{2i+1} , en en laissant au maximum un par ensemble, et en faisant en sorte qu'il en reste toujours un dans M_{2i-1} ou M_{2i+1} .

Soit $i \in \llbracket 1, n \rrbracket$. Par (P6)(1) et (P7)(1), R_{2i-1} et L_{2i+1} sont toujours matchés. De plus, les triangles pris dans T qui ont un sommet dans R_{2i-1} ou L_{2i+1} ont le sommet dans l'autre avec qui il est matché. On peut donc ajouter dans M_0 toutes les arêtes entre R_{2i-1} et L_{2i+1} qui ne sont pas déjà prises dans un triangle de T . Ainsi, il ne reste plus de sommets isolés dans les ensembles R_{2i-1} et L_{2i+1} , et par (P3), $L_1 = R_{2n+1} = \emptyset$.

Tous les sommets x_i ayant été pris dans un triangle de T , les seuls sommets qui peuvent rester dans les boîtes X_{2i} sont dans celles telles que $|X_{2i}| > 1$. On

considère donc toutes les boîtes X_{2i} telles que $|X_{2i}| > 1$. Par (P1), $|X_{2i-2}| = |X_{2i+2}| = 1$. Donc par (P6)(4), X_{2i} est couplé à M_{2i-1} et M_{2i+1} . D'après le claim 1, il y a un triangle avec x_i , x_{i+1} et $y \in M_{2i+1}$ dans T . Le triangle contenant x_{i-1} peut avoir ses deux autres sommets dans R_{2i-3} et L_{2i-1} ou dans X_{2i-4} et M_{2i-3} ou dans M_{2i-1} et X_{2i} . Par la caractérisation des triangles, aucun autre triangle de T n'a de sommets dans M_{2i-1} , X_{2i} et M_{2i+1} . Si le triangle contenant x_i n'a pas de sommet dans X_{2i} , alors on rajoute dans M_0 une arête $\{y, z\}$ avec $y \in M_{2i-1}$ et $z \in X_{2i}$ tels que $\{x_i, y, z\}$ est un triangle (qui existent bien comme vu dans la preuve du claim 1). M_{2i-1} et M_{2i+1} ont donc maintenant autant de sommets n'étant ni dans un triangle de T , ni dans une arête de M_0 , et X_{2i} en a un de moins. Si $|X_{2i}| = 2p$, alors il reste $2p - 2$ sommets dans X_{2i} à rajouter dans M_0 . On rajoute donc $p - 1$ arêtes formées par un sommet de X_{2i} et le sommet avec qui il est couplé dans M_{2i-1} et $p - 1$ arêtes formées par un sommet de X_{2i} et le sommet avec qui il est couplé dans M_{2i+1} . Il reste alors autant de sommets dans M_{2i-1} et dans M_{2i+1} et par (P7)(2) ceux restants sont tous reliés entre eux comme ils n'ont pas le même voisin dans X_{2i} . On rajoute alors autant d'arêtes que nécessaire dans M_0 , composées d'un sommet de M_{2i-1} et d'un de M_{2i+1} , n'ayant pas encore été pris, de sorte à laisser un seul sommet dans M_{2i-1} et dans M_{2i+1} . Si $|X_{2i-1}| = 2p + 1$, on procède de même mais en ne laissant cette fois ci qu'un seul sommet dans M_{2i-1} et aucun dans M_{2i+1} .

Toutes les autres boîtes M_{2i+1} , pour $i \in \llbracket 1, n - 1 \rrbracket$ ont leurs deux boîtes voisines X_{2i} et X_{2i+1} de cardinal 1, et sont donc également de cardinal 1 par (P6)(4). Et $M_1 = M_{2n+1} = \emptyset$ par (P3). Donc M_0 laisse bien des sommets uniquement dans les ensembles M_{2i+1} avec au plus un sommet pour chaque M_{2i+1} .

Par l'absurde, supposons qu'il existe $i \in \llbracket 2, n - 1 \rrbracket$ tel qu'il ne reste aucun sommet dans $M_{2i-1} \cup M_{2i+1}$. Comme si $|X_{2j}| > 1$, on a fait en sorte de toujours laisser un sommet dans M_{2j-1} , on en déduit que $|X_{2i}| = |X_{2i+2}| = 1$, et comme il ne reste plus de sommet dans M_{2i+1} , le triangle $\{x_i, x_{i+1}, y\}$ a été ajouté dans T , avec y l'unique sommet de M_{2i+1} . Or si $|X_{2i-2}| = 1$, alors de même comme M_{2i-1} n'a plus de sommet disponible, le triangle $\{x_{i-1}, x_i, y\}$ a été ajouté dans T , avec y l'unique sommet de M_{2i+1} , ce qui n'est pas possible car x_i appartiendrait à deux triangles de T , qui est un ensemble de triangles disjoints par la Proposition 1. De même, si $|X_{2i-2}| > 1$, alors par le claim 1, un triangle $\{x_{i-1}, x_i, y\}$ a été ajouté dans T , avec y un sommet de M_{2i-1} , ce qui nous fait aboutir à la même contradiction. On en conclut donc qu'il n'y a jamais de $i \in \llbracket 2, n - 1 \rrbracket$ tel que M_{2i-1} et M_{2i+1} n'aient tous les deux plus de sommets qui ne soient ni dans T ni dans M_0 .

Enfin, par (P2), on remarque que le graphe formé par G' auquel on a retiré les sommets de M_0 est bien un chemin de couleurs sans deux sommets consécutifs de la même couleur, où chaque sommet restant dans un ensemble M_{2i+1} est de couleur $i \bmod 3$.

[Retour à l'énoncé de la Proposition 2.](#)

Démonstration 3 (Preuve du Théorème 1)

1. Inclusion $\mathcal{H} \subset \mathcal{F}$.

On raisonne par contraposée. Soit G un graphe tel que $G \notin \mathcal{F}$. Par définition de $\mathcal{F}$, il existe un sous-graphe induit $G' \subseteq G$ et un entier $k \in \mathbb{N}^*$ tels que

$$G' = C_{2k+1} + K_1,$$

Autrement dit, G' est l'union disjointe d'un cycle impair de longueur $2k+1$ et d'un sommet isolé.

Observons que G' possède $2k+2$ sommets. Un couplage maximum de C_{2k+1} est de taille k (le sommet isolé ne contribue pas), donc

$$|M^*| = k.$$

De même, une couverture par sommets minimale de C_{2k+1} est de taille $k+1$, et ajouter un sommet isolé ne change pas cette taille :

$$|V_c^*| = k+1.$$

Par ailleurs,

$$\left\lfloor \frac{2k+2}{2} \right\rfloor = k+1,$$

et on obtient donc

$$|M^*| = k < k+1 = \min\left(\left\lfloor \frac{2k+2}{2} \right\rfloor, |V_c^*|\right),$$

ce qui ne respecte pas la propriété des graphes dans $\mathcal{H}$. On en déduit que $G \notin \mathcal{H}$, ce qui montre la contraposée, et donc l'inclusion $\mathcal{H} \subset \mathcal{F}$.

2. Inclusion $\mathcal{F} \subset \mathcal{H}$.

Comme $\mathcal{F}$ est stable par passage au sous-graphe induit, il suffit de montrer que pour tout graphe $G \in \mathcal{F}$, on a

$$|M^*| = \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right),$$

où M^* désigne un couplage maximum de G et V_c^* une couverture par sommets minimale.

En effet, si $G \in \mathcal{F}$, alors tous ses sous-graphes induits appartiennent également à $\mathcal{F}$. L'égalité précédente s'y applique donc pour chacun d'eux, ce qui garantit que G satisfait la propriété caractérisant $\mathcal{H}$.

Montrons d'abord l'inégalité :

$$|M^*| \leq \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right).$$

— Comme M^* est un ensemble d'arêtes deux à deux disjointes, on a immédiatement

$$|M^*| \leq \left\lfloor \frac{n}{2} \right\rfloor.$$

- Chaque arête de M^* doit être incidente à au moins un sommet de V_c^* , car V_c^* est une couverture minimale. D'où

$$|M^*| \leq |V_c^*|.$$

Ainsi,

$$|M^*| \leq \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right).$$

Établissons maintenant l'autre inégalité :

$$|M^*| \geq \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right).$$

Autrement dit, il faut montrer que

$$|M^*| \geq \left\lfloor \frac{n}{2} \right\rfloor \quad \text{ou} \quad |M^*| \geq |V_c^*|.$$

Supposons donc que

$$|M^*| < \left\lfloor \frac{n}{2} \right\rfloor,$$

et montrons sous cette hypothèse que

$$|M^*| \geq |V_c^*|.$$

Pour établir cette inégalité, commençons par décomposer l'ensemble des sommets V en trois parties disjointes :

- V_0 , l'ensemble des sommets non appariés par M^* ,
- V_1 , les sommets appariés par M^* ayant au moins un voisin dans V_0 ,
- V_2 , les sommets appariés par M^* n'ayant aucun voisin dans V_0 .

Ainsi,

$$V = V_0 \sqcup V_1 \sqcup V_2.$$

L'hypothèse

$$|M^*| < \left\lfloor \frac{n}{2} \right\rfloor,$$

se traduit par $|V_0| \geq 2$. De plus, V_0 est un stable : en effet, s'il existait une arête entre deux sommets de V_0 , on pourrait augmenter M^* en ajoutant cette arête, ce qui contredirait la maximalité du couplage.

Claim. Il n'existe pas de chemin alternant pour M^* dans G

$$(x_0, x_1, \dots, x_{2k}, x_{2k+1})$$

tel que :

- $x_0, x_{2k+1} \in V_1$,
- $x_1, \dots, x_{2k} \in V_2$,

— et les arêtes $\{x_0, x_1\}$ et $\{x_{2k}, x_{2k+1}\}$ appartiennent à M^* .

Preuve du claim. Supposons par l'absurde que ce soit le cas, et soit un tel chemin alternant

$$(x_0, x_1, \dots, x_{2k+1}),$$

où $x_0, x_{2k+1} \in V_1$, tous les sommets $x_1, \dots, x_{2k} \in V_2$, et où les arêtes $\{x_0, x_1\}$ et $\{x_{2k}, x_{2k+1}\}$ sont dans M^* . Comme x_0 et x_{2k+1} appartiennent à V_1 , chacun d'eux possède au moins un voisin dans V_0 . Supposons qu'il existe deux sommets distincts $u, v \in V_0$ tels que $\{u, x_0\} \in E$ et $\{x_{2k+1}, v\} \in E$. Alors, le chemin

$$(u, x_0, x_1, \dots, x_{2k}, x_{2k+1}, v)$$

serait un chemin augmentant pour M^* , ce qui contredirait la maximalité de ce couplage.

On en déduit donc que x_0 et x_{2k+1} n'ont qu'un seul voisin commun dans V_0 , noté x_{2k+2} . De plus, comme $|V_0| \geq 2$, il existe un autre sommet $u \in V_0$ distinct de x_{2k+2} .

Le sous-graphe induit par

$$\{x_0, x_1, \dots, x_{2k}, x_{2k+1}, x_{2k+2}, u\}$$

forme alors la réunion disjointe d'un cycle impair C_{2k+3} , obtenu en fermant le chemin de x_0 à x_{2k+1} par le sommet x_{2k+2} , et d'un sommet isolé u . En effet, x_0 et x_{2k+1} n'ont qu'un seul voisin dans V_0 qui est x_{2k+2} , tandis que les autres sommets, appartenant à V_2 , n'ont aucun voisin dans V_0 . À partir de ce cycle impair, on peut extraire un cycle impair induit. En effet, soit le cycle est déjà induit, soit il possède une corde, auquel cas cette corde le découpe en un cycle pair et un cycle impair strictement plus petit. On conclut alors par récurrence sur la longueur du cycle impair.

On obtient ainsi un sous-graphe induit isomorphe à $C_{2l+1} + K_1$, ce qui contredit l'appartenance de G à la classe $\mathcal{F}$.

Dans les deux cas, nous obtenons une contradiction.

On en déduit qu'aucun tel chemin alternant ne peut exister. $\square$

Montrons à présent qu'aucune arête de M^* ne peut avoir ses deux extrémités dans V_1 .

Procédons par l'absurde et supposons qu'il existe une telle arête. Alors celle-ci forme un chemin alternant pour M^* dont les extrémités sont dans V_1 , tous les sommets intérieurs (il n'y en a pas) sont dans V_2 , et dont les arêtes aux extrémités (il n'y en a qu'une) appartiennent à M^* . Or, on vient de montrer qu'un tel chemin n'existait pas dans G .

On conclut donc qu'aucune arête de M^* n'a ses deux sommets dans V_1 .

On peut alors partitionner le couplage M^* en deux sous-ensembles disjoints :

$$M^* = M_1 \sqcup M_2,$$

où :

- M_1 est l'ensemble des arêtes de M^* ayant un sommet dans V_1 et l'autre dans V_2 ,
- M_2 est l'ensemble des arêtes de M^* dont les deux extrémités appartiennent à V_2 .

Considérons à présent le sous-graphe induit $H = G[V_2]$. Supposons par l'absurde que H contienne un cycle impair induit. Comme V_2 est anticomplet avec V_0 , qui est non vide, en ajoutant un sommet quelconque de V_0 , on obtiendrait alors un sous-graphe induit de G étant l'union disjointe d'un cycle impair et d'un sommet isolé, ce qui contredirait l'appartenance de G à $\mathcal{F}$. On en déduit que H ne contient pas de cycle impair induit et est donc biparti.

Montrons maintenant que M_2 est un couplage maximum de H . Supposons par l'absurde que ce ne soit pas le cas. Il existe alors dans H un chemin augmentant $(x_1, x_2, \dots, x_{2k})$ pour M_2 . Comme x_1 et x_{2k} ne sont pas appariés par M_2 , ils doivent nécessairement être appariés via des arêtes de M_1 par définition de V_2 . On note ainsi :

$$\{x_0, x_1\} \in M_1 \quad \text{et} \quad \{x_{2k}, x_{2k+1}\} \in M_1,$$

où $x_0, x_{2k+1} \in V_1$.

$(x_0, x_1, \dots, x_{2k}, x_{2k+1})$ est donc un chemin alternant pour M^* dans G dont les extrémités sont dans V_1 , tous les sommets intérieurs sont dans V_2 , et dont les arêtes aux extrémités sont dans M^* . Or d'après (3), un tel chemin est impossible dans G , ce qui nous amène à une contradiction.

On en conclut donc que M_2 est bien un couplage maximum de H .

Comme H est biparti, le théorème de Kőnig assure que

$$|M_2| = |V_c^*(H)|,$$

où $V_c^*(H)$ désigne une couverture par sommets minimale de H .

Par ailleurs, on a

$$|V_1| = |M_1|,$$

car chaque sommet de V_1 est incident à une arête de M_1 , et chaque arête de M_1 contient exactement un sommet dans V_1 .

Il reste à vérifier que

$$V_c = V_1 \sqcup V_c^*(H)$$

forme une couverture par sommets de G . En effet, comme V_0 et V_2 sont anticomplets et que V_0 est stable, toute arête de G :

- soit a au moins un sommet dans V_1 , auquel cas elle est couverte par V_1 ,
- soit a ses deux extrémités dans V_2 , auquel cas elle est couverte par $V_c^*(H)$.

Ainsi,

$$|M^*| = |M_1| + |M_2| = |V_1| + |V_c^*(H)| = |V_c|.$$

En particulier, on obtient

$$|M^*| \geq |V_c^*|,$$

ce qui permet de conclure que

$$|M^*| \geq \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right).$$

En combinant avec l'inégalité inverse obtenue précédemment, on aboutit à

$$|M^*| = \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right).$$

On en déduit que tout graphe de $\mathcal{F}$ satisfait la propriété caractéristique de $\mathcal{H}$, c'est-à-dire

$$\mathcal{F} \subseteq \mathcal{H}.$$

Comme, par ailleurs, on a déjà établi l'inclusion réciproque, on conclut finalement que

$$\mathcal{F} = \mathcal{H}.$$

Retour à l'énoncé du Théorème 1.

Démonstration 4 (Preuve de la Proposition 3)

Comme les chemins de couleurs forment une classe héréditaire de graphes, il suffit de montrer qu'aucun $C_{2k+1} + K_1$ n'est un chemin de couleurs.

Supposons par l'absurde qu'il existe $k \in \mathbb{N}^*$ tel que $C_{2k+1} + K_1$ soit un chemin de couleurs G , avec une coloration

$$c : \llbracket 1, 2k+2 \rrbracket \rightarrow \llbracket 0, 2 \rrbracket.$$

Remarquons que $k \geq 2$, puisque les chemins de couleurs sont sans triangle. Si, pour tout sommet i du cycle, ses deux voisins avaient la même couleur, alors G serait 2-colorable, ce qui n'est pas possible car il contient un cycle impair.

Il existe donc un sommet $i_1 \in \llbracket 1, 2k+2 \rrbracket$ avec deux voisins i_0 et i_2 de couleurs différentes. Sans perte de généralité, supposons $c(i_0) = 0$, $c(i_1) = 1$ et $c(i_2) = 2$. On a donc $1 \leq i_0 < i_1 < i_2 \leq 2k+2$.

Soit i le sommet isolé de G . Si $c(i) = 1$, alors :

- Comme i n'est pas relié à i_2 , on a $i_2 - 1 \leq i$,
- Comme i n'est pas relié à i_0 , on a $i \leq i_0 + 1$.

On en déduit $i_2 - 1 \leq i \leq i_0 + 1 \leq i_1 \leq i_2 - 1$, donc $i = i_1$, ce qui est absurde. Ainsi, $c(i) \neq 1$. Par symétrie, supposons $c(i) = 0$. Alors :

- Comme i n'est pas relié à i_1 , on a $i_1 - 1 \leq i$,
- Comme i n'est pas relié à i_2 , on a $i \leq i_2 + 1$.

Soit i_3 le deuxième voisin de i_0 . Alors $c(i_3) \in \{1, 2\}$. Si $c(i_3) = 2$, alors $i_3 < i_0$, et comme i_3 n'est pas relié à i , on a $i \leq i_3 + 1$, donc :

$$i \leq i_3 + 1 \leq i_0 \leq i_1 - 1 \leq i,$$

donc $i = i_0$, ce qui est absurde. Donc $c(i_3) = 1$. On a alors :

$$i_0 < i_3, \quad i_2 - 1 \leq i_3, \quad i_3 - 1 \leq i.$$

Soit i_4 le deuxième voisin de i_3 . Alors $c(i_4) \in \{0, 2\}$. Si $c(i_4) = 2$, alors $i_3 < i_4$, donc :

$$i_1 < i_2 \leq i_3 + 1 \leq i_4.$$

Si $i_4 = i_1 + 1$, alors $i_4 = i_2$, ce qui est absurde. Donc $i_1 + 2 \leq i_4$, ce qui implique que i_1 et i_4 sont voisins, contradiction. Ainsi, $c(i_4) = 0$, et $i_4 < i_3$. Comme i_4 n'est pas relié à i_1 , on a $i_1 - 1 \leq i_4$.

Si $i_2 < i_4$, alors comme $i \leq i_2 + 1$ et $i_3 - 1 \leq i$, on obtient :

$$i_3 - 1 \leq i \leq i_2 + 1 \leq i_4 \leq i_3 - 1,$$

donc $i = i_4$, ce qui est absurde. Donc $i_4 < i_2$, et ainsi i_2 et i_4 ne sont pas voisins, donc $G \neq C_5 + K_1$.

Le sommet i_4 a donc un deuxième voisin i_5 , avec $c(i_5) \in \{1, 2\}$.

- Si $c(i_5) = 1$, alors $i_4 < i_5$, et comme i_5 n'est pas relié à i_0 , on a $i_5 \leq i_0 + 1$. On obtient :

$$i_1 - 1 \leq i_4 \leq i_5 - 1 \leq i_0 \leq i_1 - 1,$$

donc $i_0 = i_4$, contradiction.

- Donc $c(i_5) = 2$. Alors $i_5 < i_4$, et comme i_5 n'est pas relié à i , on a $i \leq i_5 + 1$. Donc,

$$i_5 + 1 \leq i_4 \leq i_3 - 1 \leq i \leq i_5 + 1,$$

donc $i = i_4$, ce qui est encore absurde.

On conclut donc qu'aucun $C_{2k+1} + K_1$ ne peut être un chemin de couleurs.

[Retour à l'énoncé de la Proposition 3.](#)

Démonstration 5 (Preuve de la Proposition 4)

Soit G un chemin de couleurs à n sommets tel que deux sommets consécutifs ne soient jamais de la même couleur. Soit S^* un stable maximum de G .

1^{er} cas : il y a au moins un sommet de chaque couleur dans S^* . Soit $i_0 = \min(S^*)$. Sans perte de généralité, supposons $c(i_0) = 0$. Soit $i_1 \in S^*$ tel que $c(i_1) = 1$ et $i_2 \in S^*$ tel que $c(i_2) = 2$.

Comme S^* est un stable, $i_1 - 1 \leq i_0$, et comme $i_0 = \min(S^*)$, on a $i_0 < i_1$, donc $i_0 \leq i_1 - 1 \leq i_0$, ce qui implique $i_1 = i_0 + 1$.

De même, $i_1 < i_2$, et comme i_1 et i_2 ne sont pas voisins, $i_2 - 1 \leq i_1$, donc $i_2 = i_1 + 1$.

Supposons par l'absurde que $|S^*| \geq 5$. Alors il existe au moins deux autres sommets dans S^* , notés i_3 et i_4 avec $i_3 < i_4$. Alors :

$$i_0 + 2 \leq i_4, \quad i_1 + 2 \leq i_4, \quad i_2 + 2 \leq i_4.$$

Quelle que soit la couleur de i_4 , il est voisin de l'un des sommets i_0 , i_1 ou i_2 , ce qui contredit le fait que S^* soit un stable. On en déduit :

$$3 \leq |S^*| \leq 4.$$

Donc :

$$|S^*| \leq \left\lfloor \frac{|S^*|}{2} \right\rfloor + 2 \leq \left\lfloor \frac{n}{2} \right\rfloor + 2.$$

2^{ème} cas : tous les sommets de S^* sont de la même couleur. Alors, comme il n'y a jamais deux sommets consécutifs de la même couleur, on a :

$$|S^*| \leq \left\lceil \frac{n}{2} \right\rceil \leq \left\lfloor \frac{n}{2} \right\rfloor + 2.$$

3^{ème} cas : S^* contient exactement deux couleurs. Sans perte de généralité, supposons qu'il s'agit des couleurs 0 et 1. On note :

$$S_0 = S^* \cap V_0, \quad S_1 = S^* \cap V_1.$$

Soit $i_0 = \min(S_0)$ et $i_1 = \max(S_1)$. Comme i_0 et i_1 ne sont pas voisins, $i_1 \leq i_0 - 1$. Donc :

$$S_1 \subset V_1 \cap \llbracket 1, i_0 + 1 \rrbracket, \quad S_0 \subset V_0 \cap \llbracket i_0, n \rrbracket.$$

Or, comme deux sommets consécutifs ne peuvent avoir la même couleur, on a :

$$|V_1 \cap \llbracket 1, i_0 + 1 \rrbracket| \leq \left\lceil \frac{i_0 + 1}{2} \right\rceil, \quad |V_0 \cap \llbracket i_0, n \rrbracket| \leq \left\lceil \frac{n - i_0 + 1}{2} \right\rceil.$$

D'où :

$$\begin{aligned}
|S^*| &\leq |S_1| + |S_0| \\
&\leq |V_1 \cap \llbracket 1, i_0 + 1 \rrbracket| + |V_0 \cap \llbracket i_0, n \rrbracket| \\
&\leq \left\lceil \frac{i_0 + 1}{2} \right\rceil + \left\lceil \frac{n - i_0 + 1}{2} \right\rceil \\
&\leq \frac{i_0 + 2}{2} + \frac{n - i_0 + 2}{2} = \frac{n}{2} + 2
\end{aligned}$$

Et en appliquant de chaque côté la partie entière qui est croissante :

$$|S^*| \leq \left\lfloor \frac{n}{2} \right\rfloor + 2.$$

[Retour à l'énoncé de la Proposition 4.](#)

Démonstration 6 (Preuve du Théorème 2)

D'après la Proposition 2, il existe un couplage partiel M_0 dans G' tel que le sous-graphe G'' , obtenu en supprimant les sommets couplés par M_0 , est un chemin de couleurs dans lequel deux sommets consécutifs ne sont jamais de la même couleur.

Comme $\mathcal{G}$ est l'ensemble des chemins de couleurs, $G'' \in \mathcal{G}$. Or, d'après la Proposition 3, on a $\mathcal{G} \subset \mathcal{F}$, et le Théorème 1 donne l'égalité $\mathcal{F} = \mathcal{H}$. Ainsi, $G'' \in \mathcal{H}$.

Par définition de $\mathcal{H}$, on a alors l'égalité suivante appliquée à G'' :

$$|M^*| = \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right),$$

où n désigne le nombre de sommets de G'' , M^* un couplage maximum de G'' , et V_c^* une couverture minimale de sommets de G'' .

Par ailleurs, pour tout graphe on a l'égalité $|V_c^*| = n - |S^*|$, avec S^* un stable maximum de G'' . Or, d'après la Proposition 4, on a :

$$|S^*| \leq \left\lfloor \frac{n}{2} \right\rfloor + 2,$$

ce qui donne :

$$|V_c^*| \geq n - \left(\left\lfloor \frac{n}{2} \right\rfloor + 2\right) = \left\lceil \frac{n}{2} \right\rceil - 2.$$

On en déduit que :

$$|M^*| = \min\left(\left\lfloor \frac{n}{2} \right\rfloor, |V_c^*|\right) \geq \left\lceil \frac{n}{2} \right\rceil - 2.$$

Le nombre de sommets non appariés par M^* dans G'' est donc au plus :

$$n - 2|M^*| \leq n - 2\left(\left\lceil \frac{n}{2} \right\rceil - 2\right) = n - 2\left\lceil \frac{n}{2} \right\rceil + 4.$$

Par définition de la partie supérieure, on a toujours :

$$n - 2\left\lceil \frac{n}{2} \right\rceil \leq 0,$$

et donc le nombre de sommets non appariés dans G'' est au plus 4. En combinant ce couplage M^* avec le couplage partiel M_0 , on obtient un couplage dans G' laissant au plus 4 sommets non appariés.

[Retour à l'énoncé du Théorème 2.](#)

A.2 Extrait de code C++

Code 1 (Code du Programme 1)

```

1 #include <bits/stdc++.h>
2 #include <boost/graph/adjacency_list.hpp>
3 #include <boost/graph/max_cardinality_matching.hpp>
4 using namespace std;
5 typedef boost::adjacency_list<boost::vecS, boost::vecS,
6                               boost::undirectedS>
7     Graph;
8 typedef vector<int> vi;
9 typedef pair<int, int> pii;
10 #define sz(v) (int)v.size()
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(0);
15     int choix = 2;
16     int n;
17     while (cin >> n) {
18         vi a(n);
19         int p = 0;
20         for (int i = 0; i != n; i++) {
21             cin >> a[i];
22             p += a[i];
23         }
24         vector<vector<vector<pii>>> s(3);
25         vector<vi> boite(3);
26         s[0] = vector<vector<pii>>(n);
27         boite[0] = vi(n);
28         for (int i = 0; i != n; i++) {
29             boite[0][i] = 2 * i;
30         }
31         for (int i = 0; i != n - 1; i++) {
32             s[0][i].emplace_back(0, i + 1);
33             s[0][i + 1].emplace_back(0, i);
34         }
35         s[1] = vector<vector<pii>>(n - 1);
36         boite[1] = vi(n - 1);
37         for (int i = 0; i != n - 1; i++) {
38             s[1][i].emplace_back(0, i);
39             s[1][i].emplace_back(0, i + 1);
40             s[0][i].emplace_back(1, i);
41             s[0][i + 1].emplace_back(1, i);

```

```

42     boite[1][i] = 2 * i + 1;
43 }
44 s[2] = vector<vector<pii>>(2 * p);
45 int j1 = 0;
46 for (int i = 0; i != n; i++) {
47     if (a[i] != 0) {
48         s[2][j1].emplace_back(0, i);
49         s[0][i].emplace_back(2, j1);
50         boite[2].push_back(2 * i - 1);
51         s[2][j1].emplace_back(2, j1 + 1);
52         s[2][j1 + 1].emplace_back(2, j1);
53         j1++;
54         s[2][j1].emplace_back(0, i);
55         s[0][i].emplace_back(2, j1);
56         boite[2].push_back(2 * i + 1);
57         j1++;
58     }
59 }
60 for (int i = 0; i != 3; i++) {
61     for (int j = 0; j != sz(s[i]); j++) {
62         for (int k = 0; k != 3; k++) {
63             for (int l = 0; l != sz(s[k]); l++) {
64                 if ((boite[i][j] + 3 <= boite[k][l] &&
65                     (boite[k][l] - boite[i][j]) % 3 == 2) ||
66                     (boite[i][j] + 2 == boite[k][l] &&
67                     (boite[i][j] % 2 == 0 ||
68                     (i == 1 && k == 2 && l % 2 == 0) ||
69                     (i == 2 && k == 1 && j % 2 == 1)))) {
70                     s[i][j].emplace_back(k, l);
71                     s[k][l].emplace_back(i, j);
72                 }
73             }
74         }
75     }
76 }
77 int tmax = 0;
78 for (int i = 0; i != n; i++) {
79     tmax++;
80     if (a[i] == 0) {
81         i++;
82     }
83 }
84 vector<vector<vector<pii>>> s2(2);
85 s2[0] = vector<vector<pii>>(n - 1);
86 s2[1] = vector<vector<pii>>(p);
87 for (int i = 0; i != n - 2; i++) {
88     s2[0][i].emplace_back(0, i + 1);
89     s2[0][i + 1].emplace_back(0, i);
90 }
91 j1 = 0;

```

```

122     for (int i = 0; i != n; i++) {
123         if (a[i] != 0) {
124             if (i > 0) {
125                 s2[1][j1].emplace_back(0, i - 1);
126                 s2[0][i - 1].emplace_back(1, j1);
127             }
128             if (i < n - 1) {
129                 s2[1][j1].emplace_back(0, i);
130                 s2[0][i].emplace_back(1, j1);
131             }
132             j1++;
133         }
134     }
135     vector<pii> current_sol;
136     vector<pii> best_sol;
137     int k = 0;
138     vector<vector<bool>> vus2(3);
139     int res = INT_MIN;
140     int best_k = -1;
141     vector<tuple<int, int, int, int>> best_matching_of_sol;
142     function<bool(void)> complete_with_matching =
143         [&](void) -> bool {
144             vus2[0] = vector<bool>(n);
145             vus2[1] = vector<bool>(n - 1);
146             vus2[2] = vector<bool>(2 * p);
147             for (auto u : current_sol) {
148                 auto [i, j] = u;
149                 if (i == 0) {
150                     vus2[0][j] = true;
151                     vus2[0][j + 1] = true;
152                     vus2[1][j] = true;
153                 } else {
154                     vus2[0][s[2][2 * j][0].second] = true;
155                     vus2[2][2 * j] = true;
156                     vus2[2][2 * j + 1] = true;
157                 }
158             }
159             int nb_sommets = 0;
160             map<pii, int> dic;
161             vector<pii> sommets;
162             for (int i = 0; i != 3; i++) {
163                 for (int j = 0; j != sz(s[i]); j++) {
164                     if (!vus2[i][j]) {
165                         dic[{i, j}] = nb_sommets++;
166                         sommets.emplace_back(i, j);
167                     }
168                 }
169             }
170             Graph g(sz(dic));
171             for (auto x : dic) {

```

```

142     auto [i, j] = x.first;
143     for (auto y : s[i][j]) {
144         auto [i2, j2] = y;
145         if (!vus2[i][j] && !vus2[i2][j2] &&
146             (i < i2 || (i == i2 && j < j2))) {
147             boost::add_edge(x.second, dic[{i2, j2}], g);
148         }
149     }
150 }
151 std::vector<boost::graph_traits<Graph>::vertex_descriptor>
152     mate(num_vertices(g));
153 edmonds_maximum_cardinality_matching(g, &mate[0]);
154 int matching_max = matching_size(g, &mate[0]);
155 if (matching_max - 2 * k > res) {
156     best_k = k;
157     res = matching_max - 2 * k;
158     best_sol = current_sol;
159     best_matching_of_sol.clear();
160     for (int i = 0; i != sz(mate); i++) {
161         if (mate[i] !=
162             boost::graph_traits<Graph>::null_vertex() &&
163             i < (int)mate[i]) {
164             best_matching_of_sol.emplace_back(
165                 sommets[i].first, sommets[i].second,
166                 sommets[mate[i]].first,
167                 sommets[mate[i]].second);
168         }
169     }
170     if (2 * n - 1 + 2 * p - 3 * tmax - 2 * res - k < 2) {
171         return true;
172     }
173 }
174 return false;
175 };
176 vector<vi> vus(2);
177 vus[0] = vi(n - 1);
178 vus[1] = vi(p);
179 int t;
180 function<pii(int, int)> suivant = [&](int i, int j) -> pii {
181     if (i == 1 && j == p - 1)
182         return {-1, -1};
183     if (i == 0 && j == n - 2)
184         return {1, 0};
185     return {i, j + 1};
186 };
187 function<int(int, int)> h = [&](int i, int j) -> int {
188     if (i == 1)
189         return p - j;
190     return n - 1 - j + p;
191 };

```

```

192     function<bool(int, int)> stable = [&](int i,
193                                         int j) -> bool {
194         if (t == 0) {
195             return complete_with_matching();
196         }
197         if (i == -1 || h(i, j) < t)
198             return false;
199         pii w = suivant(i, j);
200         if (!vus[i][j]) {
201             current_sol.emplace_back(i, j);
202             t--;
203             for (auto v : s2[i][j]) {
204                 vus[v.first][v.second]++;
205             }
206             if (stable(w.first, w.second)) {
207                 return true;
208             }
209             current_sol.pop_back();
210             t++;
211             for (auto v : s2[i][j]) {
212                 vus[v.first][v.second]--;
213             }
214         }
215         return stable(w.first, w.second);
216     };
217     t = tmax;
218     while (!stable(0, 0)) {
219         if (2 * n - 1 + 2 * p - 3 * tmax - 2 * res - k - 3 < 0) {
220             // if it is not possible to improve the solution
221             break;
222         }
223         t--;
224         k++;
225     }
226     if (choix == 0) {
227         cout << "total : " << 2 * n - 1 + 2 * p - res - 2 * tmax
228             << endl;
229         cout << "triangles max : " << tmax << endl;
230         cout << "triangles : " << tmax - best_k << endl;
231         cout << "aretes : " << res + 2 * best_k << endl;
232         cout << "sommets isolés : "
233             << 2 * n - 1 + 2 * p - 3 * tmax - 2 * res - best_k
234             << endl;
235         for (auto x : best_sol) {
236             cout << x.first << ' ' << x.second << endl;
237         }
238         for (auto x : best_matching_of_sol) {
239             cout << get<0>(x) << ' ' << get<1>(x) << ' '
240                 << get<2>(x) << ' ' << get<3>(x) << endl;
241         }

```



```

242     break;
243 } else if (choix == 1) {
244     if (best_k >= 1) {
245         if (best_k == 2) {
246             cout << "!!!\n";
247         } else if (best_k > 2) {
248             cout << "this is impossible\n";
249         }
250         cout << n << endl;
251         for (int i = 0; i != n; i++) {
252             cout << a[i] << ' ';
253         }
254         cout << endl;
255         cout << "nombre triangles : " << tmax - best_k << endl;
256         cout << "nombre de sommets isolés : "
257             << 2 * n - 1 + 2 * p - 3 * tmax - 2 * res - best_k
258             << endl;
259     }
260 } else if (choix == 2) {
261     cout << 2 * n - 1 + 2 * p - res - 2 * tmax << endl;
262 }
263 }
264 return 0;
265 }

```

Listing 1.1 – Algorithme naïf en C++

[Retour à la description du Programme 1.](#)

Code 2 (Code du Programme 2)

```

1  #include <bits/stdc++.h>
2  #include <boost/graph/adjacency_list.hpp>
3  #include <boost/graph/max_cardinality_matching.hpp>
4  using namespace std;
5  typedef boost::adjacency_list<boost::vecS, boost::vecS,
6                               boost::undirectedS>
7      Graph;
8  typedef vector<int> vi;
9  typedef pair<int, int> pii;
10 #define sz(v) (int)v.size()
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(0);
15     int choix = 1;
16     int n;
17     while (cin >> n) {
18         vi a(n);
19         int p = 0;
20         for (int i = 0; i != n; i++) {

```

```

21     cin >> a[i];
22     p += a[i];
23 }
24 vector<vector<vector<pii>>> s(3);
25 vector<vi> boite(3);
26 s[0] = vector<vector<pii>>(n);
27 boite[0] = vi(n);
28 for (int i = 0; i != n; i++) {
29     boite[0][i] = 2 * i;
30 }
31 for (int i = 0; i != n - 1; i++) {
32     s[0][i].emplace_back(0, i + 1);
33     s[0][i + 1].emplace_back(0, i);
34 }
35 s[1] = vector<vector<pii>>(n - 1);
36 boite[1] = vi(n - 1);
37 for (int i = 0; i != n - 1; i++) {
38     s[1][i].emplace_back(0, i);
39     s[1][i].emplace_back(0, i + 1);
40     s[0][i].emplace_back(1, i);
41     s[0][i + 1].emplace_back(1, i);
42     boite[1][i] = 2 * i + 1;
43 }
44 s[2] = vector<vector<pii>>(2 * p);
45 int j1 = 0;
46 for (int i = 0; i != n; i++) {
47     if (a[i] != 0) {
48         s[2][j1].emplace_back(0, i);
49         s[0][i].emplace_back(2, j1);
50         boite[2].push_back(2 * i - 1);
51         s[2][j1].emplace_back(2, j1 + 1);
52         s[2][j1 + 1].emplace_back(2, j1);
53         j1++;
54         s[2][j1].emplace_back(0, i);
55         s[0][i].emplace_back(2, j1);
56         boite[2].push_back(2 * i + 1);
57         j1++;
58     }
59 }
60 for (int i = 0; i != 3; i++) {
61     for (int j = 0; j != sz(s[i]); j++) {
62         for (int k = 0; k != 3; k++) {
63             for (int l = 0; l != sz(s[k]); l++) {
64                 if ((boite[i][j] + 3 <= boite[k][l] &&
65                     (boite[k][l] - boite[i][j]) % 3 == 2) ||
66                     (boite[i][j] + 2 == boite[k][l] &&
67                     (boite[i][j] % 2 == 0 ||
68                     (i == 1 && k == 2 && l % 2 == 0) ||
69                     (i == 2 && k == 1 && j % 2 == 1)))) {
70                     s[i][j].emplace_back(k, l);

```

```

71         s[k][1].emplace_back(i, j);
72     }
73 }
74 }
75 }
76 }
77 vector<vector<vector<pii>>> s2(2);
78 s2[0] = vector<vector<pii>>(n - 1);
79 s2[1] = vector<vector<pii>>(p);
80 for (int i = 0; i != n - 2; i++) {
81     s2[0][i].emplace_back(0, i + 1);
82     s2[0][i + 1].emplace_back(0, i);
83 }
84 j1 = 0;
85 for (int i = 0; i != n; i++) {
86     if (a[i] != 0) {
87         if (i > 0) {
88             s2[1][j1].emplace_back(0, i - 1);
89             s2[0][i - 1].emplace_back(1, j1);
90         }
91         if (i < n - 1) {
92             s2[1][j1].emplace_back(0, i);
93             s2[0][i].emplace_back(1, j1);
94         }
95         j1++;
96     }
97 }
98 vector<pii> triangles_pris;
99 int tmax = 0;
100 j1 = 0;
101 for (int i = 0; i != n; i++) {
102     tmax++;
103     if (a[i] == 0) {
104         triangles_pris.emplace_back(0, i);
105         i++;
106         if (a[i] == 1) {
107             j1++;
108         }
109     } else {
110         triangles_pris.emplace_back(1, j1++);
111     }
112 }
113 vector<vector<bool>> vus2(3);
114 vus2[0] = vector<bool>(n);
115 vus2[1] = vector<bool>(n - 1);
116 vus2[2] = vector<bool>(2 * p);
117 for (auto u : triangles_pris) {
118     auto [i, j] = u;
119     if (i == 0) {
120         vus2[0][j] = true;

```

```

121     vus2[0][j + 1] = true;
122     vus2[1][j] = true;
123 } else {
124     vus2[0][s[2][2 * j][0].second] = true;
125     vus2[2][2 * j] = true;
126     vus2[2][2 * j + 1] = true;
127 }
128 }
129 int nb_sommets = 0;
130 map<pii, int> dic;
131 vector<pii> sommets;
132 for (int i = 0; i != 3; i++) {
133     for (int j = 0; j != sz(s[i]); j++) {
134         if (!vus2[i][j]) {
135             dic[{i, j}] = nb_sommets++;
136             sommets.emplace_back(i, j);
137         }
138     }
139 }
140 Graph g(sz(dic));
141 for (auto x : dic) {
142     auto [i, j] = x.first;
143     for (auto y : s[i][j]) {
144         auto [i2, j2] = y;
145         if (!vus2[i][j] && !vus2[i2][j2] &&
146             (i < i2 || (i == i2 && j < j2))) {
147             boost::add_edge(x.second, dic[{i2, j2}], g);
148         }
149     }
150 }
151 std::vector<boost::graph_traits<Graph>::vertex_descriptor>
152     mate(num_vertices(g));
153 edmonds_maximum_cardinality_matching(g, &mate[0]);
154 int m = matching_size(g, &mate[0]);
155 vector<tuple<int, int, int, int>> best_matching;
156 for (int i = 0; i != sz(mate); i++) {
157     if (mate[i] !=
158         boost::graph_traits<Graph>::null_vertex() &&
159         i < (int)mate[i]) {
160         best_matching.emplace_back(
161             sommets[i].first, sommets[i].second,
162             sommets[mate[i]].first, sommets[mate[i]].second);
163     }
164 }
165 if (choix == 0) {
166     cout << "total : " << 2 * n - 1 + 2 * p - m - 2 * tmax
167         << endl;
168     cout << "triangles : " << tmax << endl;
169     cout << "aretes : " << m << endl;
170     cout << "sommets isolés : "

```

```

171         << 2 * n - 1 + 2 * p - 3 * tmax - 2 * m << endl;
172     for (auto x : triangles_pris) {
173         cout << x.first << ' ' << x.second << endl;
174     }
175     for (auto x : best_matching) {
176         cout << get<0>(x) << ' ' << get<1>(x) << ' '
177             << get<2>(x) << ' ' << get<3>(x) << endl;
178     }
179     break;
180 } else if (choix == 1) {
181     cout << 2 * n - 1 + 2 * p - m - 2 * tmax << endl;
182 }
183 }
184 return 0;
185 }

```

Listing 1.2 – Algorithme d'approximation en C++

[Retour à la description du Programme 2.](#)

Code 3 (Code du Programme 3)

```

1  #include <bits/stdc++.h>
2  #include <sys/types.h>
3  #include <sys/wait.h>
4  #include <unistd.h>
5  using namespace std;
6
7  const int N = 25;
8  bool vus[1 << N];
9
10 int rev(int i) {
11     int res = 0;
12     int p = 32 - __builtin_clz(i);
13     while (p-- > 0) {
14         res <<= 1;
15         res ^= i & 1;
16         i >>= 1;
17     }
18     return res;
19 }
20
21 void random_test(int n) {
22     int r = rand() % (1 << (n - 2));
23     int p = (1 << (n - 1)) ^ (r << 1) ^ 1;
24     vus[max(p, rev(p))] = true;
25 }
26
27 void random_test_variable_size(int n) {
28     while (n > 2 && rand() % 2) {
29         n--;

```

```

30     }
31     random_test(n);
32 }
33
34 void test_all(int n) {
35     for (int i = 0; i != (1 << (n - 2)); i++) {
36         int p = (1 << (n - 1)) ^ (i << 1) ^ 1;
37         vus[max(p, rev(p))] = true;
38     }
39 }
40
41 void test_all_from(int n, int deb) {
42     for (int i = (deb >> 1) ^ (1 << (n - 2)); i != (1 << (n - 2));
43         i++) {
44         int p = (1 << (n - 1)) ^ (i << 1) ^ 1;
45         vus[max(p, rev(p))] = true;
46     }
47 }
48
49 void start_test(void) {
50     for (int i = 0; i != (1 << N); i++) {
51         if (vus[i]) {
52             int p = 32 - __builtin_clz(i);
53             cout << p << endl;
54             while (p-- > 0) {
55                 cout << (i >> p & 1) << ' ';
56             }
57             cout << endl;
58         }
59     }
60 }
61
62 std::string run_program(const char *program,
63                        const std::string &input) {
64     int pipe_in[2]; // parent -> child
65     int pipe_out[2]; // child -> parent
66
67     if (pipe(pipe_in) == -1 || pipe(pipe_out) == -1) {
68         perror("pipe failed");
69         exit(1);
70     }
71
72     pid_t pid = fork();
73     if (pid == -1) {
74         perror("fork failed");
75         exit(1);
76     }
77
78     if (pid == 0) {
79         // ---- Enfant ----

```

```

80     dup2(pipe_in[0],
81          STDIN_FILENO); // stdin du programme = pipe_in[0]
82     dup2(pipe_out[1],
83          STDOUT_FILENO); // stdout du programme = pipe_out[1]
84
85     close(pipe_in[1]);
86     close(pipe_out[0]);
87     close(pipe_in[0]);
88     close(pipe_out[1]);
89
90     execl(program, program, (char *)nullptr);
91     perror("execl failed");
92     exit(1);
93 } else {
94     // ---- Parent ----
95     close(pipe_in[0]);
96     close(pipe_out[1]);
97
98     // Ecrire dans stdin du child
99     write(pipe_in[1], input.c_str(), input.size());
100    close(pipe_in[1]); // Signaler EOF
101
102    // Lire stdout du child
103    char buffer[256];
104    std::string output;
105    ssize_t n;
106    while ((n = read(pipe_out[0], buffer, sizeof(buffer) - 1)) >
107           0) {
108        buffer[n] = '\0';
109        output += buffer;
110    }
111    close(pipe_out[0]);
112
113    waitpid(pid, nullptr, 0);
114    return output;
115 }
116 }
117
118 void comparer(const char *cmd1, const char *cmd2) {
119     for (int i = 0; i != (1 << N); i++) {
120         if (vus[i]) {
121             int p = 32 - __builtin_clz(i);
122             string s = to_string(p) + "\n";
123             while (p-- > 0) {
124                 s += to_string(i >> p & 1) + " ";
125             }
126             s.push_back('\n');
127             string out1 = run_program(cmd1, s);
128             string out2 = run_program(cmd2, s);
129             int a = stoi(out1);

```

```

130     int b = stoi(out2);
131     if (b - a == 2) {
132         cerr << s << out1 << out2;
133     }
134 }
135 }
136 }
137
138 int main() {
139     const int nb_iterations = 1000000;
140     const int n = 20;
141     assert(n <= N);
142     int deb = 0b1101101001001001001;
143     int choix = 3;
144     bool comp = true;
145     const char cmd1[] = "./algo_naif_3";
146     const char cmd2[] = "./algo_approximation";
147     if (choix == 4) {
148         assert(deb < (1 << n));
149     }
150     if (choix == 0) {
151         test_all(n);
152     } else if (choix == 1) {
153         for (int i = 0; i != nb_iterations; i++) {
154             random_test(n);
155         }
156     } else if (choix == 2) {
157         for (int i = 0; i != nb_iterations; i++) {
158             random_test_variable_size(n);
159         }
160     } else if (choix == 3) {
161         for (int i = 2; i != n + 1; i++) {
162             test_all(i);
163         }
164     } else if (choix == 4) {
165         test_all_from(n, deb);
166     }
167     if (comp) {
168         comparer(cmd1, cmd2);
169     } else {
170         start_test();
171     }
172     return 0;
173 }

```

Listing 1.3 – Générateur de tests en C++

[Retour à la description du Programme 3.](#)

Code 4 (Code du Programme 4)


```

1 #include <bits/stdc++.h>
2 #include <boost/graph/adjacency_list.hpp>
3 #include <boost/graph/max_cardinality_matching.hpp>
4 using namespace std;
5 typedef boost::adjacency_list<boost::vecS, boost::vecS,
6                               boost::undirectedS>
7     Graph;
8 typedef vector<int> vi;
9 typedef pair<int, int> pii;
10 #define sz(v) (int)v.size()
11
12 int min_vertex_cover(vector<pii> &edges, vector<bool> &vus,
13                     int n, int i) {
14     if (i == sz(edges)) {
15         return 0;
16     }
17     pii e = edges[i];
18     if (vus[e.first] || vus[e.second]) {
19         return min_vertex_cover(edges, vus, n, i + 1);
20     }
21     vus[e.first] = true;
22     int a = min_vertex_cover(edges, vus, n, i + 1);
23     vus[e.first] = false;
24     vus[e.second] = true;
25     int b = min_vertex_cover(edges, vus, n, i + 1);
26     vus[e.second] = false;
27     return min(a, b) + 1;
28 }
29
30 int matching_max(vector<pii> &edges, int n) {
31     Graph g(n);
32     for (auto e : edges) {
33         boost::add_edge(e.first, e.second, g);
34     }
35     std::vector<boost::graph_traits<Graph>::vertex_descriptor>
36         mate(num_vertices(g));
37     edmonds_maximum_cardinality_matching(g, &mate[0]);
38     int m = matching_size(g, &mate[0]);
39     return m;
40 }
41
42 bool is_path(vector<pii> &edges, int n) {
43     vi d(n);
44     for (auto e : edges) {
45         d[e.first]++;
46         d[e.second]++;
47         if (d[e.first] == 2 && d[e.second] == 2) {
48             return false;
49         }
50     }

```

```

51     int deg1 = 0;
52     for (int i = 0; i != n; i++) {
53         if (d[i] == 1) {
54             deg1++;
55         } else if (d[i] != 2) {
56             return false;
57         }
58     }
59     return deg1 == 2;
60 }
61
62 int main() {
63     srand(time(NULL));
64     int debut = 1;
65     int fin = 18;
66     bool find_path = true;
67     vi chemin(debut);
68     while (sz(chemin) <= fin) {
69         int i = 0;
70         int n = sz(chemin);
71         while (chemin[i] == 2) {
72             chemin[i++] = 0;
73         }
74         if (i == n - 1) {
75             cout << n << endl;
76             chemin = vi(++n);
77         } else {
78             chemin[i]++;
79         }
80         int t = 1;
81         for (int i = 0; i != n - 1; i++) {
82             if ((chemin[i] + 1) % 3 == chemin[i + 1]) {
83                 t *= 2;
84             }
85         }
86         int k = 0;
87         while (k != t) {
88             int p = 0;
89             vector<pii> edges;
90             for (int i = 0; i < n; i++) {
91                 for (int j = i + 1; j < n; j++) {
92                     if ((chemin[i] + 1) % 3 == chemin[j]) {
93                         if (j == i + 1) {
94                             if (k >> p++ & 1) {
95                                 continue;
96                             }
97                         }
98                         edges.emplace_back(i, j);
99                     }
100                 }

```

```

101     }
102     if (find_path && is_path(edges, n)) {
103         cout << "chemin de taille : " << n << endl;
104         for (auto x : chemin) {
105             cout << x << ' ';
106         }
107         cout << endl;
108         cout << k << endl;
109     }
110     vector<bool> vus(n);
111     int m = matching_max(edges, n);
112     if (n - 2 * m >= 2) {
113         int v = min_vertex_cover(edges, vus, n, 0);
114         if (v != m) {
115             cout << n << endl;
116             for (auto x : chemin) {
117                 cout << x << ' ';
118             }
119             cout << endl;
120             cout << k << endl;
121         }
122     }
123     k++;
124 }
125 }
126 return 0;
127 }

```

Listing 1.4 – Tests sur les chemins de couleurs en C++

[Retour à la description du Programme 4.](#)